

ИДЗ_3_АКОС

Комкова Полина Дмитриевна БПИ244

Вариант 14

Условие

В магазине работают **два отдела**, каждый отдел обладает уникальным ассортиментом. В **каждом** отделе работает **один продавец**.

В магазин ходят исключительно забывчивые **покупатели**, поэтому каждый покупатель носит с собой **список из K товаров**, которые желает купить. Покупатель приобретает товары точно в том порядке, в каком они записаны в его списке. Число товаров в списке **от одного до пяти**. При этом товары в списке расположены в случайном порядке, что заставляет покупателя многократно переходить от отдела к отделу, если это требуется для совершения покупок.

Продавец может обслужить **только одного** покупателя за раз. Покупатель, вставший в очередь, **засыпает** пока не дойдет до продавца. Продавец **засыпает**, если в его отделе нет покупателей, и просыпается, если появится хотя бы один.

Создать **многопроцессное приложение**, моделирующее работу магазина в течение рабочего дня. Каждый покупатель и продавец задаются отдельным процессом

Структура файлов

```
INW_3/
├── Program_4-6                # Первая программа
│   ├── main.c
│   └── ...
├── Program_7-8                # Вторая программа
│   ├── shop.c
│   └── ...
├── Program_9                  # Третья программа
│   ├── shop.c
│   └── ...
└── Program_10                 # Четвертая программа
    ├── shop.c
```

Файлы программ (структура +- одна для всех программ)

- **shop.h**: Описание программы. Сюда вынесены все объявления функций, глобальных переменных и тд
- **main.c/shop.c**: Общие функции, инициализация памяти, семафоров, обработка сигналов, очистка ресурсов
- **seller1.c**: Обслуживание очереди к первому отделу. Тут происходит ожидание покупателей, обработка заказов и тд
- **seller2.c**: То же самое для второго отдела
- **customer.c**: Клиент с генерацией списков покупок, становление в очереди и ожиданием обслуживания
- **observer.c**: Мониторит, собирает инфу от всех процессов через именованные каналы. Выводит данные о работе системы

Запуск кода

Program_4-6

```
gcc main.c seller1.c seller2.c customer.c -o shop -pthread -lrt
```

1 терминал: ./shop <x>, где x – количество покупателей

Program_7-8

- **Запуск**

```
gcc -c shop.c -o shop.o -lrt -lpthread  
gcc seller1.c shop.o -o seller1 -lrt -lpthread  
gcc seller2.c shop.o -o seller2 -lrt -lpthread  
gcc customer.c shop.o -o customer -lrt -lpthread
```

1 терминал: ./seller1

2 терминал: ./seller2

```
3 терминал: ./customer
n терминал: ./customer
```

- **Остановка**

Ctrl+C в терминалах продавцов

Program_9

- **Запуск**

```
gcc -c shop.c -o shop.o -lrt -lpthread
gcc seller1.c shop.o -o seller1 -lrt -lpthread
gcc seller2.c shop.o -o seller2 -lrt -lpthread
gcc customer.c shop.o -o customer -lrt -lpthread
gcc observer.c shop.o -o observer -lrt -lpthread
```

```
1 терминал: ./observer
2 терминал: ./seller1
3 терминал: ./seller2
4 терминал: ./customer
n терминал: ./customer
```

- **Остановка**

Ctrl+C в терминалах продавцов
Затем Ctrl+C в терминале наблюдателя

Program_10

- **Запуск**

```
gcc -c shop.c -o shop.o -lrt -lpthread
gcc seller1.c shop.o -o seller1 -lrt -lpthread
gcc seller2.c shop.o -o seller2 -lrt -lpthread
gcc customer.c shop.o -o customer -lrt -lpthread
```

```
1 терминал: ./observer
2 терминал: ./observer
3 терминал: ./seller1
4 терминал: ./seller2
```

```
5 терминал: ./customer
n терминал: ./customer
```

- **Остановка**

```
Ctrl+C в терминалах продавцов
Затем Ctrl+C в терминале наблюдателей
```

Описание имеющихся объектов

- **Родитель (Программа_4-6):** *главный управляющий*, который инициализирует разделяемую память и семафоры, создает дочерние процессы, обрабатывает сигналы (Ctrl+C), делает завершение и очистку ресурсов
- **Продавцы (2 процесса):** семафоры продавцов ожидают уведомлений. Когда в очереди появляется покупатель, они берут индекс покупателя из соотв. очереди в разделяемой памяти, обслуживают, и затем ставят `sem_post` на семафор конкретного покупателя
В случае Программы_7-8 еще извлекают ID покупателя из очереди
В случае Программы_9 и 10 при ключевых событиях (старт, встать в очередь, начать/закончить обслуживание, завершение) формируют сообщения и вызывают `send_to_observer(...)`
- **Покупатели (N процессов):** формируют случайный список покупок, встают в очередь нужного отдела, то есть записывают свой индекс в массив-очередь, ставят `sem_post` продавцу и затем `sem_wait` на собственный семафор ожидания обслуживания. После обслуживания покупатель либо продолжает покупки, либо завершает работу
В случае Программы_9 и 10, так же, как и продавцы, формируют сообщения для наблюдателя
- **Независимые процессы (Программа_7-8):** каждый процесс при старте открывает/создает именованные семафоры. При завершении закрывает дескрипторы и последний процесс выполняет `sem_unlink / shm_unlink`
- **Наблюдатель (Программа_9):** отдельный файл `observer`, который создает/открывает именованный `OBSERVER_FIFO (/tmp/shop_observer_fifo)` и читает из него поток сообщений. Он выводит сообщения в свою консоль со временем и `pid`. Обрабатывает `SIGINT/SIGTERM` и при завершении закрывает `FIFO` и удаляет его
В случае Программы_10 наблюдатель регистрирует в `shop->observer_pids[]` вызовом `register_observer(getpid())`, создает/открывает `/tmp/shop_observer_<pid>` для чтения

Структуры

- **Shared memory (POSIX):**
 - **Программа_4-6:** `shop_t` в общей памяти содержит очереди, `front/rear`, флаги (`day_over`, `shutdown_flag`) и массив неименованных семафоров
 - **Программа_7-8:** `shop_t` содержит очереди, `front/rear`, `day_over`, `next_customer_id`, `active_processes`
- **Семафоры:**
 - **Программа_4-6:** *неименованные* семафоры и массив семафоров покупателей в SHM (инициализация `sem_init(..., pshared=1)`)
 - **Программа_7-8:** *именованные* семафоры (`sem_open`): `SEM_MUTEX`, `SEM_SELLER1`, `SEM_SELLER2`, `SEM_DAY_OVER` и динамические `SEM_CUSTOMER_<id>`
- **Защита структур:** общий мьютекс (`mutex`) защищает доступ к очередям и счетчику `active_processes` (**Программа_7-8**)
- **Очистка:** в **Программе_9 и 10** `cleanup_resources()` закрывает файловые дескриптор, и если текущий процесс последний (по `shop->active_processes`), то удаляет SHM и семафоры. Наблюдатель дополнительно делает `unlink` FIFO при выходе
- Поток сообщений идет по FIFO, синхронизация очередей и семафоры остаются через POSIX-shm и именованные семафоры (как в Программе_7-8)

Алгоритм работы (Программа_4-6)

1. Родитель создает `shm_open + ftruncate + mmap` и инициализирует структуру `shop_t`. Также инициализирует **НЕИМЕНОВАННЫЕ** семафоры (`sem_init(..., pshared=1, ...)`)
2. Потом он создает **2 процесса-продавца** и **N процессов-покупателей**
3. Каждый покупатель:
 - Генерирует **список** покупок;
 - Для каждой позиции проверяет флаг `day_over`. Если магазин открыт, захватывает `mutex`, **записывает свой индекс в очередь**, обновляет `rear`, `sem_post` соответствующему продавцу и `sem_wait` на своем семафоре
 - После `sem_wait` покупатель проверяет `shutdown_flag/day_over`. Если сработал аварийный выход, **покидает магазин**, иначе считает товар купленным и **переходит к следующему**
4. Каждый продавец:
 - Ожидает `sem_wait(&sellerX_sem)`, затем под `mutex` проверяет, **есть ли покупатель в его очереди**

- Извлекает клиента из очереди, освобождает `mutex` , **имитирует обслуживание**, `sem_post` на семафор этого клиента
- При установленных флагах `day_over` и пустой очереди продавец **завершает** работу

5. Завершение **нормальным** способом:

Когда все N покупателей завершили, родитель ставит `day_over = 1` , пробуждает продавцов, ждет завершения продавцов и очищает ресурсы (`sem_destroy` , `munmap` , `shm_unlink`)

6. При **прерывании** (`SIGINT` / `SIGTERM`) только **родитель** обрабатывает сигнал:

выставляет `shutdown_flag` и `day_over` и делает `sem_post` на всех семафорах продавцов и покупателей, чтобы разблокировать дочерние процессы и дать им корректно завершиться. Дети игнорируют `SIGINT` / `SIGTERM` , чтобы сигнал обрабатывался один раз

Тестовое покрытие

- Удачный запуск без прерываний И с прерыванием `Ctrl+C`

<pre> Select polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Program_4-6 polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_4-6\$./shop 2 Открытие магазина... Число покупателей: 2 Продавец 1 начал работу Продавец 2 начал работу ----- Попкупатель 0: Мой список покупок: 2 1 ----- Попкупатель 0: Жду в очереди 2 отдела для товара 2 Продавец 2 обслуживает попкуателя 0 ----- Попкупатель 1: Мой список покупок: 1 ----- Попкупатель 1: Жду в очереди 1 отдела для товара 1 Продавец 1 обслуживает попкуателя 1 Продавец 1 закончил обслуживание попкуателя 1 Попкупатель 1: Купил товар из отдела 1 Попкупатель 1: Завершил покупки Продавец 2 закончил обслуживание попкуателя 0 Попкупатель 0: Купил товар из отдела 2 Попкупатель 0: Жду в очереди 1 отдела для товара 1 Продавец 1 обслуживает попкуателя 0 Продавец 1 закончил обслуживание попкуателя 0 Попкупатель 0: Купил товар из отдела 1 Попкупатель 0: Завершил покупки ----- Заккрытие магазина... Продавец 1 завершил работу Продавец 2 завершил работу Магазин закрыт! polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_4-6\$ </pre>	<pre> polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Program_4-6 polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_4-6\$./shop 2 Открытие магазина... Число покупателей: 2 Продавец 1 начал работу Продавец 2 начал работу ----- Попкупатель 0: Мой список покупок: 2 ----- Попкупатель 0: Жду в очереди 2 отдела для товара 2 ----- Попкупатель 1: Мой список покупок: 1 2 2 ----- Попкупатель 1: Жду в очереди 1 отдела для товара 1 Продавец 2 обслуживает попкуателя 0 Продавец 1 обслуживает попкуателя 1 ^C Принят сигнал 2, отключаемся... Попкупатель 0: Магазин закрывается, ухажу Попкупатель 1: Магазин закрывается, ухажу Попкупатель 0: Завершил покупки Попкупатель 1: Завершил покупки ----- Заккрытие магазина... Продавец 1 закончил обслуживание попкуателя 1 Продавец 1 завершил работу Продавец 2 закончил обслуживание попкуателя 0 Продавец 2 завершил работу Магазин закрыт! polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_4-6\$ </pre>
--	---

Критерии

4-6 баллов

1. ☒ Представить сценарий решаемой задачи поясняющий, каким образом исходные сущности и их поведение отображаются в процессы и их взаимодействие:
У меня есть покупатели - дочерние процессы, продавцы - еще два дочерних, обмен через POSIX-shm, синхронизация через неименованные семафоры в разделяемой памяти
2. ☒ Множество процессов взаимодействуют с использованием неименованных POSIX семафоров:
Здесь использованы (sem_init с pshared=1)
3. ☒ Обмен данными ведется через разделяемую память в стандарте POSIX:
Используется (shm_open + mmap)
4. ☒ Реализовать завершение программы в соответствии с условием задачи, а также предусмотреть корректное завершение всей программы по прерыванию с клавиатуры при подаче соответствующего сигнала:
При Ctrl+C родитель централизованно завершает систему, и все дочерние процессы корректно освобождают ресурсы и завершаются, семафоры удаляются
5. ☒ Каждый из процессов выводит в общий поток данных информацию необходимую для понимания его работы в соответствии с выполняемой им ролевой функцией (очевидно, видно в тестовом покрытии)
6. ☒ В программе предусмотреть удаление семафоров и разделяемой памяти по ее завершению любым из способов:
Реализовано в родителе после ожидания дочерних процессов

Алгоритм работы (Программа_7-8)

- Любой процесс при старте вызывает `init_shared_resources()` :
 - `shm_open` с `O_CREAT|O_EXCL` , затем `mmap`
 - `sem_open` для именованных семафоров
- После `mmap` процесс под `mutex` увеличивает `shop->active_processes` , а первый процесс при необходимости инициализирует поля очередей
- **Покупатель:**
 - Получает уникальный `customer_id = shop->next_customer_id++`
 - Создает свой именованный семафор `SEM_CUSTOMER_<id>` (через `get_customer_sem(..., 1)`);
 - Для каждой позиции в списке: вставляет свой `customer_id` в нужную очередь, `sem_post` продавцу, затем ждет на своем семафоре с таймаутом `sem_timedwait`

При обслуживании получает сигнал и продолжает, при `day_over / shutdown_flag` выходит

- **Продавец:**

- В цикле ожидает `sellerX_sem`
- Проверяет очередь: если есть клиент, то извлекает `customer_id`, обслуживает и открывает `SEM_CUSTOMER_<id>` через `get_customer_sem(..., 0)`, `sem_post` и `sem_close`

- **Завершение:**

- Любой процесс при получении `SIGINT/SIGTERM` выставляет `shutdown_flag` и корректно вызывает `cleanup_resources()`
- `cleanup_resources()` уменьшает `active_processes`

Если счетчик стал 0, это означает, что текущий процесс *последний*: он выполняет `shm_unlink` и `sem_unlink` для именованных семафоров, а также перебирает и `sem_unlink` всех потенциальных `SEM_CUSTOMER_<i>`

Все процессы выполняют `sem_close`, `munmap`, `close(shm_fd)`

Тестовое покрытие

The image displays four terminal windows arranged in a 2x2 grid, showing the output of a program. The top-left window shows the execution of `./seller1`, where a seller initializes a shop, serves two customers, and then receives a signal to finish. The top-right window shows `./customer` for customer 0, who lists purchases, waits in a queue, and buys items from two different departments. The bottom-left window shows `./seller2`, where a second seller serves two more customers. The bottom-right window shows customer 1's activity, including a purchase list and waiting in a queue. All windows show the completion of the program with cleanup messages.

```
polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Program_7-8
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$ ./seller1
Продавец 1: Инициализировал магазин
Продавец 1 начал работу
-----
Продавец 1 обслуживает покупателя 0
Продавец 1 закончил обслуживание покупателя 0
Продавец 1 обслуживает покупателя 1
Продавец 1 закончил обслуживание покупателя 1
^C
Процесс 72999: Получен сигнал 2, завершаем работу...
-----
Продавец 1 завершил работу
Процесс 72999: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$

polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Program_7-8
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$ ./customer
Покупатель 0 начал покупки
-----
Покупатель 0: Мой список покупок: 2 1 2
-----
Покупатель 0: Встал в очередь 2 отдела для товара 2
Покупатель 0: Купил товар из отдела 2
Покупатель 0: Встал в очередь 1 отдела для товара 1
Покупатель 0: Купил товар из отдела 1
Покупатель 0: Встал в очередь 2 отдела для товара 2
Покупатель 0: Купил товар из отдела 2
-----
Покупатель 0: Завершил покупки
-----
Процесс 73013: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$

polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Program_7-8
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$ ./seller2
Продавец 2 начал работу
-----
Продавец 2 обслуживает покупателя 0
Продавец 2 закончил обслуживание покупателя 0
Продавец 2 обслуживает покупателя 1
Продавец 2 закончил обслуживание покупателя 1
Продавец 2 обслуживает покупателя 0
Продавец 2 закончил обслуживание покупателя 0
^C
Процесс 73006: Получен сигнал 2, завершаем работу...
-----
Продавец 2 завершил работу
Процесс 73006: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$

polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Program_7-8
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$ ./customer
Покупатель 1 начал покупки
-----
Покупатель 1: Мой список покупок: 2 1
-----
Покупатель 1: Встал в очередь 2 отдела для товара 2
Покупатель 1: Купил товар из отдела 2
Покупатель 1: Встал в очередь 1 отдела для товара 1
Покупатель 1: Купил товар из отдела 1
-----
Покупатель 1: Завершил покупки
-----
Процесс 73020: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_7-8$
```

Критерии

1. ☒ Множество процессов взаимодействуют с использованием именованных POSIX семафоров:
Реализованы (`sem_open/sem_close/sem_unlink`), в т.ч. динамические `SEM_CUSTOMER_<id>`
2. ☒ Обмен данными ведется через разделяемую память в стандарте POSIX:
Используются `shm_open/ftruncate/mmap/shm_unlink`
3. ☒ Для завершения приложения использовать те же подходы, что и в предыдущих решениях:
Обработка `SIGINT/SIGTERM`, а также `cleanup_resources()` уменьшает `active_processes` и последний процесс удаляет `shared/sem`
4. ☒ Каждый из процессов выводит в свою консоль информацию необходимую для понимания его работы в соответствии с выполняемой им ролевой функцией:
Продавец/покупатель печатают свои действия

Алгоритм работы (Программа_9)

К предыдущему приложению добавила отдельный процесс **наблюдатель**. Он запускается отдельно в своей консоли и получает текстовые сообщения от остальных процессов через именованный канал. Наблюдатель собирает информацию о действиях продавцов и покупателей

- Покупатель при вхождении в очередь `sem_post(sellerX_sem)` и сообщение наблюдателю "Встал в очередь X отдела"
- Продавец при начале обслуживания отправляет "Обслуживает покупателя N", по окончании "Закончил обслуживание покупателя N"
- Наблюдатель **печатает** присланные сообщения
- Любой процесс при Ctrl+C обработает сигнал (`SIGINT/SIGTERM`) и вызовет `cleanup_resources()`
- `cleanup_resources()` уменьшит `shop->active_processes`. Если это последний процесс, то он удалит `shm`, семафоры и FIFO
- Наблюдатель при завершении закрывает и `unlink(OBSERVER_FIFO)`

Тестовое покрытие

```
polia@LAPTOP-KJ14OKRT: ~/AKOC/IDZ_3/Prog... x
r1
Продавец 1: Инициализировал магазин
Продавец 1 начал работу
-----
Продавец 1 обслуживает покупателя 0
Продавец 1 закончил обслуживание покупателя 0
Продавец 1 обслуживает покупателя 1
Продавец 1 закончил обслуживание покупателя 1
Продавец 1 обслуживает покупателя 0
Продавец 1 закончил обслуживание покупателя 0
^C
Процесс 13487: Получен сигнал 2, завершаем работу...
-----
Продавец 1 завершил работу
Процесс 13487: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_9$

polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Prog... x
r2
Продавец 2 начал работу
-----
Продавец 2 обслуживает покупателя 0
Продавец 2 закончил обслуживание покупателя 0
Продавец 2 обслуживает покупателя 0
Продавец 2 закончил обслуживание покупателя 0
Продавец 2 обслуживает покупателя 1
Продавец 2 закончил обслуживание покупателя 1
^C
Процесс 13488: Получен сигнал 2, завершаем работу...
-----
Продавец 2 завершил работу
Процесс 13488: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_9$

Покупатель 0 начал покупки
-----
Покупатель 0: Мой список покупок: 2 2 1 1
Покупатель 0: Встал в очередь 2 отдела для товара 2
Покупатель 0: Купил товар из отдела 2
Покупатель 0: Встал в очередь 2 отдела для товара 2
Покупатель 0: Встал в очередь 1 отдела для товара 1
Покупатель 0: Купил товар из отдела 1
Покупатель 0: Встал в очередь 1 отдела для товара 1
Покупатель 0: Купил товар из отдела 1
Покупатель 0: Завершил покупки
-----
Процесс 13489: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_9$

Покупатель 1 начал покупки
-----
Покупатель 1: Мой список покупок: 1 2
Покупатель 1: Встал в очередь 1 отдела для товара 1
Покупатель 1: Купил товар из отдела 1
Покупатель 1: Встал в очередь 2 отдела для товара 2
Покупатель 1: Купил товар из отдела 2
Покупатель 1: Завершил покупки
-----
Процесс 13490: Очистка завершена
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_9$

[13:58:13] SELLER2 (PID: 13488): Обслуживает покупат
еля 0
[13:58:14] SELLER2 (PID: 13488): Закончил обслуживан
ие покупателя 0
[13:58:14] CUSTOMER (PID: 13489): Встал в очередь 1
отдела
[13:58:14] SELLER1 (PID: 13487): Обслуживает покупат
еля 0
[13:58:14] CUSTOMER (PID: 13490): Начал покупки
[13:58:14] CUSTOMER (PID: 13490): Список покупок: 2
товаров
[13:58:14] CUSTOMER (PID: 13490): Встал в очередь 1
отдела
[13:58:16] SELLER1 (PID: 13487): Закончил обслуживан
ие покупателя 0
[13:58:16] CUSTOMER (PID: 13489): Встал в очередь 1
отдела
[13:58:16] SELLER1 (PID: 13487): Обслуживает покупат
еля 1
[13:58:18] SELLER1 (PID: 13487): Закончил обслуживан
ие покупателя 1
[13:58:18] SELLER1 (PID: 13487): Обслуживает покупат
еля 0
[13:58:18] CUSTOMER (PID: 13490): Встал в очередь 2
отдела
[13:58:18] SELLER2 (PID: 13488): Обслуживает покупат
еля 1
[13:58:19] SELLER2 (PID: 13488): Закончил обслуживан
ие покупателя 1
[13:58:19] CUSTOMER (PID: 13490): Завершил покупки
[13:58:20] SELLER1 (PID: 13487): Закончил обслуживан
ие покупателя 0
[13:58:20] CUSTOMER (PID: 13489): Завершил покупки
[13:58:42] SELLER1 (PID: 13487): Завершил работу
[13:58:45] SELLER2 (PID: 13488): Завершил работу
^C
Процесс 13486: Получен сигнал 2, завершаем работу...
Наблюдатель завершил работу
polia@LAPTOP-KJ14OKRT:~/AKOC/IDZ_3/Program_9$
```

Критерии

9 баллов

- ✓ Для завершения приложения использовать те же подходы, что и в предыдущих решениях:
Те же подходы, как в предыдущих решениях: SIGINT/SIGTERM, cleanup_resources() и удаление именованных объектов последним процессом
- ✓ Все процессы запускаются независимо друг от друга. Процесс-наблюдатель получает информацию через именованный канал или очередь сообщений. Ранее разработанные процессы, дополнительно с отображением в своей консоли, передают порождаемую ими информацию наблюдателю:
Все так и реализовано. Обмен с наблюдателем через именованный FIFO (mkfifo, open, read/write)

Алгоритм работы (Программа_10)

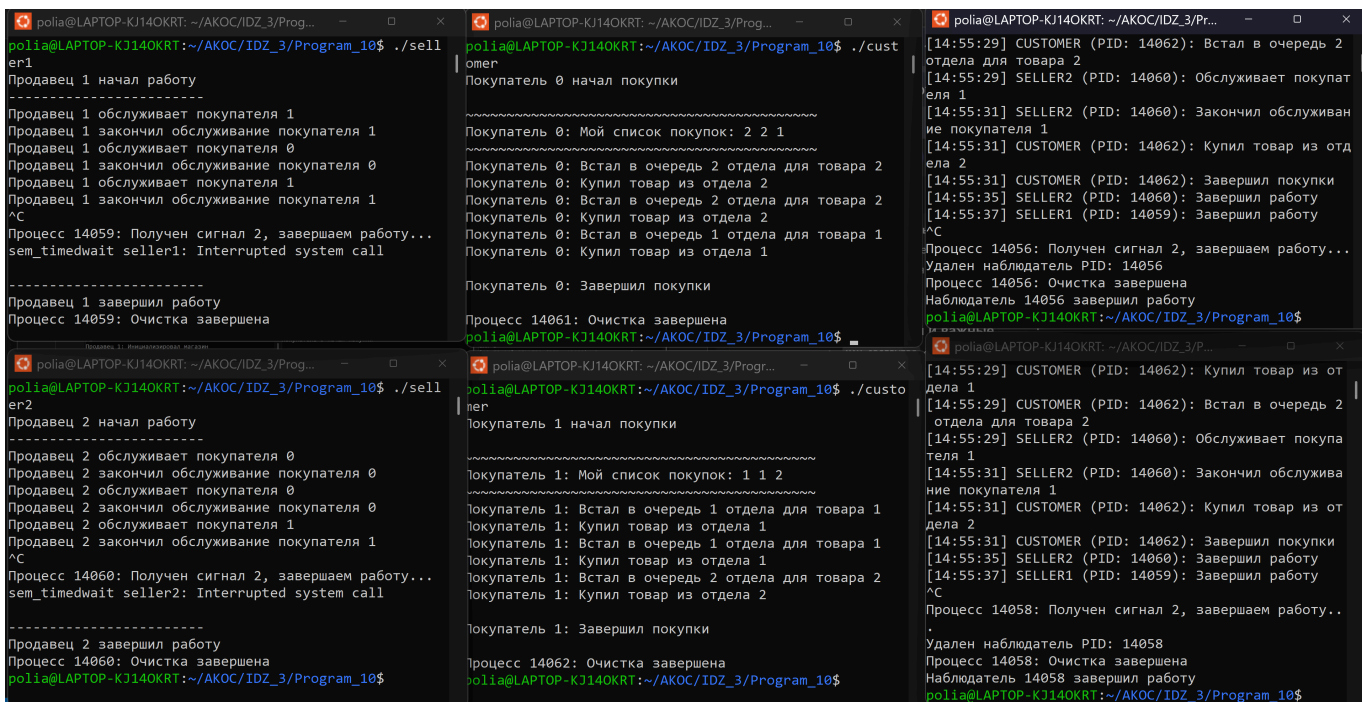
Тут я добавила механизм подключения нескольких независимых наблюдателей. Каждый наблюдатель запускается в своей консоли, регистрируется в общей SHM и получает сообщения от процессов через собственный именованный FIFO

```
/tmp/shop_observer_<pid>
```

Все наблюдатели работают одинаково и согласованно. При отправке сообщения процессы рассылают его всем **зарегистрированным** наблюдателям

- Наблюдатель **при старте**:
 1. Вызывает `init_shared_resources()` (создает/открывает SHM и семафоры)
 2. Увеличивает `shop->active_processes` и вызывает `register_observer(getpid())`;
 3. Формирует свое имя FIFO `/tmp/shop_observer_<pid>`, вызывает `mkfifo(...)` и открывает его для чтения (`open(..., O_RDONLY | O_NONBLOCK)`)
- При **выходе** наблюдатель:
 1. Закрывает FIFO, `unlink(fifo_name)`, вызывает `unregister_observer(getpid())`, уменьшает `shop->active_processes` и вызывает `cleanup_resources()`
- `cleanup_resources()`:
 - Уменьшает `active_processes`; если это **последний** процесс, удаляет SHM и именованные семафоры и очищает семафоры покупателей, а также удаляет все наблюдательские FIFO, но корректно по `observer_pids[]`

Тестовое покрытие



The screenshot displays three terminal windows running a program with multiple processes and observers. The windows show the following output:

- Window 1 (Left):** Shows the execution of `./seller1`. It logs the start of the seller, the service of customers 1 and 0, and the completion of the seller's work. It also shows the receipt of a signal 2 and the completion of the seller's work.
- Window 2 (Middle):** Shows the execution of `./custo`. It logs the start of the customer, the purchase of goods from departments 2 and 1, and the completion of the customer's work. It also shows the receipt of a signal 2 and the completion of the customer's work.
- Window 3 (Right):** Shows the execution of `./custo`. It logs the start of the customer, the purchase of goods from departments 2 and 1, and the completion of the customer's work. It also shows the receipt of a signal 2 and the completion of the customer's work.

Критерии

-  Для завершения приложения использовать те же подходы, что и в предыдущих решениях:
Те же подходы, как в предыдущих решениях: SIGINT/SIGTERM, `cleanup_resources()` и удаление именованных объектов последним процессом
-  Все наблюдатели работают одинаково и согласованно. Вариант их подключения к процессам, решающим основную задачу, является произвольным:
Все наблюдатели запускаются независимо и работают согласованно: доставка сообщений каждому наблюдателю через индивидуальный FIFO
`/tmp/shop_observer_<pid>`